

Java Threads | How to create a thread in Java

Multithreading is a fundamental concept in Java programming, allowing developers to execute multiple tasks concurrently within a single program. **Threads** are lightweight processes that run within the context of a larger process, enabling efficient utilization of system resources and enhancing application responsiveness. In this section, we will explore how to create threads in Java, covering various approaches, best practices.

In Java, threads are represented by instances of the `Thread` class or by implementing the `Runnable` interface. The `Thread` class provides built-in support for multithreading, while the `Runnable` interface defines a single method, `run()` that contains the code to be executed by the thread. By implementing the `Runnable` interface, we can decouple the task from the thread itself, promoting better code organization and reusability.

There are the following two ways to create a thread:

- By Extending Thread Class
- By Implementing Runnable Interface

Thread Class

The simplest way to create a thread in Java is by extending the `Thread` class and overriding its `run()` method. `Thread` class provide constructors and methods to create and perform operations on a thread. `Thread` class extends `Object` class and implements `Runnable` interface.

Constructors of Thread Class

- `Thread()`
- `Thread(String name)`
- `Thread(Runnable r)`
- `Thread(Runnable r, String name)`

Thread Class Methods

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread. JVM calls the run() method on the thread.
3. **public void sleep(long milliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. **public void join():** waits for a thread to die.
5. **public void join(long milliseconds):** waits for a thread to die for the specified milliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.
10. **public Thread currentThread():** returns the reference of currently executing thread.
11. **public int getId():** returns the id of the thread.
12. **public Thread.State getState():** returns the state of the thread.
13. **public boolean isAlive():** tests if the thread is alive.
14. **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. **public void suspend():** is used to suspend the thread(deprecated).
16. **public void resume():** is used to resume the suspended thread(deprecated).
17. **public void stop():** is used to stop the thread(deprecated).
18. **public boolean isDaemon():** tests if the thread is a daemon thread.
19. **public void setDaemon(boolean b):** marks the thread as daemon or user thread.
20. **public void interrupt():** interrupts the thread.
21. **public boolean isInterrupted():** tests if the thread has been interrupted.
22. **public static boolean interrupted():** tests if the current thread has been interrupted.

By Implementing Runnable Interface

Another approach to creating threads in Java is by implementing the Runnable interface. The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named run(). This approach is preferred when we want to separate the task from the thread itself, promoting better encapsulation and flexibility.

public void run(): is used to perform action for a thread.

Starting a Thread

The **start() method** of the Thread class is used to start a newly created thread. It performs the following tasks:

- A new thread starts (with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.

Thread Creation

1) Creating Thread by Extending Thread Class

File Name: Multi.java

```
class Multi extends Thread{
    public void run(){
        System.out.println("thread is running...");
    }
    public static void main(String args[]){
        Multi t1=new Multi();
        t1.start();
    }
}
```

Output:

```
thread is running...
```

In this example, we define a custom thread by extending the Thread class and overriding the run() method with our desired functionality. We then instantiate the thread and call its start() method to begin execution.

2) Java Thread Example by implementing Runnable interface

FileName: Multi3.java

```
class Multi3 implements Runnable{
    public void run(){
```

```
System.out.println("thread is running...");
}

public static void main(String args[]){
Multi3 m1=new Multi3();
Thread t1=new Thread(m1); // Using the constructor Thread(Runnable r)
t1.start();
}
}
```

Output:

```
thread is running...
```

In this example, we define a class `MyRunnable` that implements the `Runnable` interface and provides the task logic inside its `run()` method. We then create a new `Thread` object, passing an instance of `MyRunnable` to its constructor, and start the thread using the `start()` method.

If we are not extending the `Thread` class, the class object would not be treated as a thread object. So, we need to explicitly create the `Thread` class object. We are passing the object of our class that implements `Runnable` so that the class's `run()` method may execute.

3) Using the Thread Class: Thread(String Name)

We can directly use the `Thread` class to spawn new threads using the constructors defined above.

FileName: `MyThread1.java`

```
public class MyThread1
{
// Main method
public static void main(String args[])
{
// creating an object of the Thread class using the constructor Thread(String name
)
```

```
Thread t= new Thread("My first thread");

// the start() method moves the thread to the active state
t.start();
// getting the thread name by invoking the getName() method
String str = t.getName();
System.out.println(str);
}
}
```

Output:

```
My first thread
```

4) Using the Thread Class: Thread(Runnable r, String name)

Observe the following program.

FileName: MyThread2.java

```
public class MyThread2 implements Runnable
{
    public void run()
    {
        System.out.println("Now the thread is running ...");
    }

    // main method
    public static void main(String argsv[])
    {
        // creating an object of the class MyThread2
        Runnable r1 = new MyThread2();

        // creating an object of the class Thread using Thread(Runnable r, String name)
        Thread th1 = new Thread(r1, "My new thread");

        // the start() method moves the thread to the active state
```

```
th1.start();

// getting the thread name by invoking the getName() method
String str = th1.getName();
System.out.println(str);
}
}
```

Output:

```
My new thread
Now the thread is running ...
```

Create a thread in Java MCQ

1. Which of the following is a correct way to create a thread in Java?

1. Implementing the Runnable interface
2. Extending the Thread class
3. Both a and b
4. Neither a nor b

[Show Answer](#)[Workspace](#)

2. What method must be overridden when implementing the Runnable interface?

1. start()
2. run()
3. init()
4. execute()

[Show Answer](#)[Workspace](#)

3. What is the correct sequence to start a thread using the Runnable interface?

1. Create an instance of Thread class and call start()
2. Call run() method directly
3. Create an instance of Runnable class and call start()
4. Create an instance of Thread class and call run()

[Show Answer](#)[Workspace](#)

4. What is the state of a thread when the start() method is called?

1. New
2. Runnable
3. Running
4. Waiting

[Show Answer](#)[Workspace](#)

5. How can we create a daemon thread in Java?

1. By calling setDaemon(true) method on a thread object
2. By overriding the run() method
3. By extending the Thread class
4. By implementing the Runnable interface

[Show Answer](#)[Workspace](#)

Next Topic[Thread Scheduler](#)[← prev](#)[next →](#)

Learn Important Tutorial



JavaScript

HTML



Database



PHP



C++



React

B.Tech / MCA



DBMS



Data
Structures





Operating
System



Computer
Network



Compiler
Design



Computer
Organization



Discrete
Mathematics



Ethical
Hacking



Computer
Graphics



Web
Technology



Software
Engineering



Cyber
Security



Automata



C
Programming



C++



Java



.Net



Python



Programs



Control
System



Data
Warehouse

Preparation



Aptitude



Reasoning



Verbal
Ability



Interview
Questions



Company
Questions